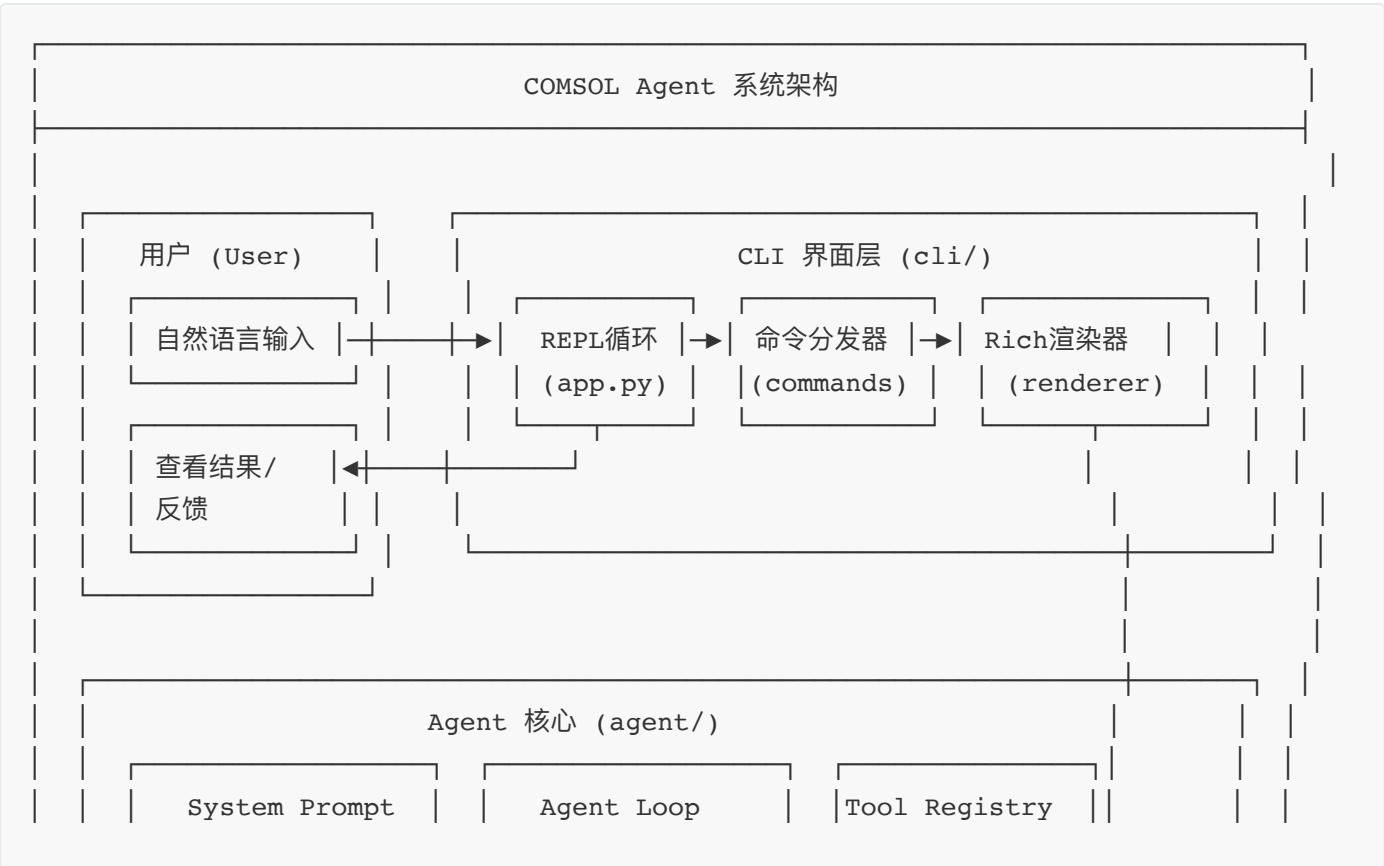


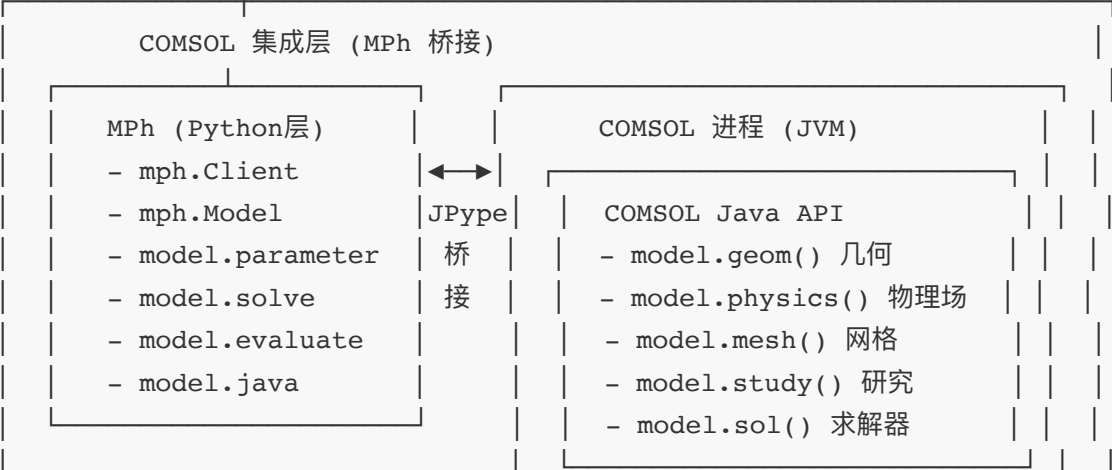
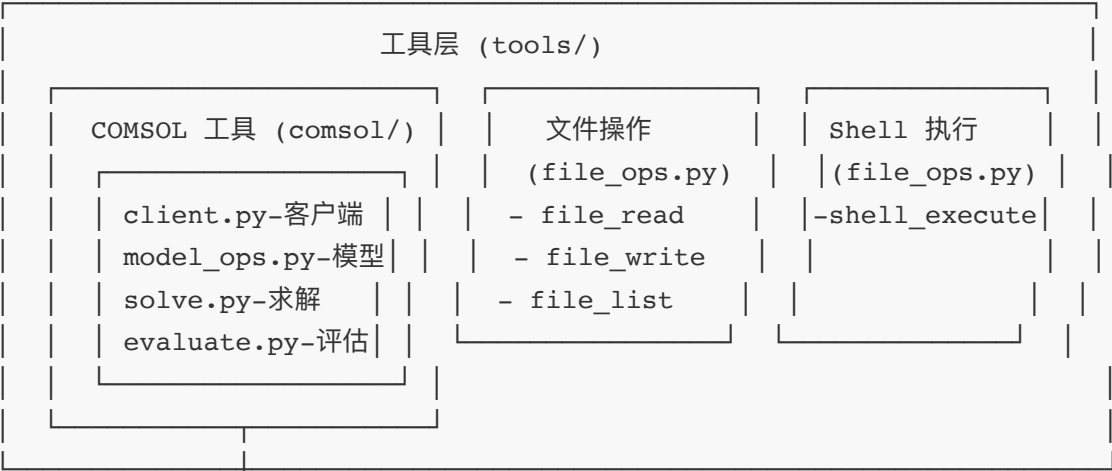
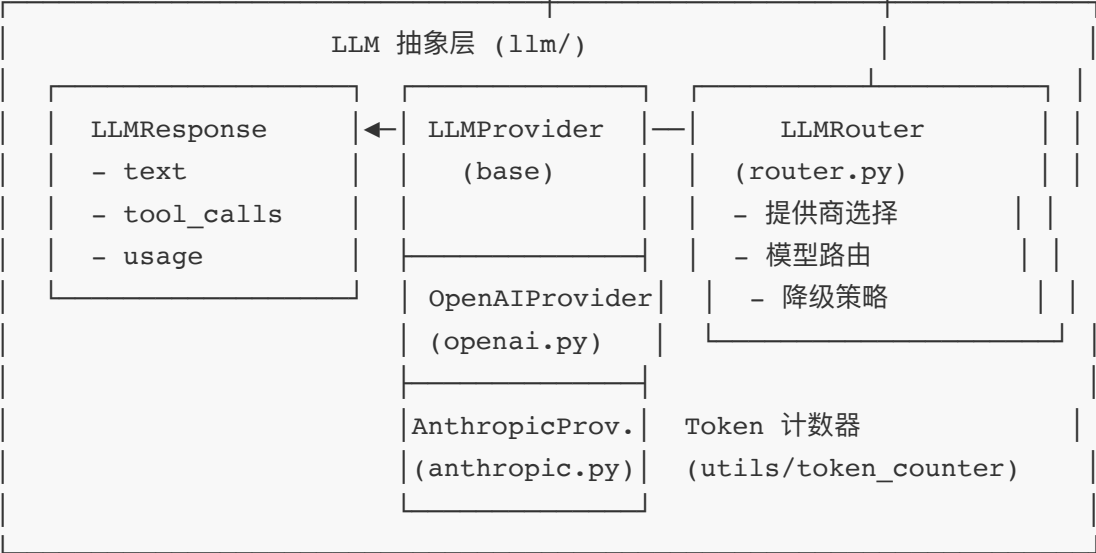
COMSOL Agent 技术架构文档

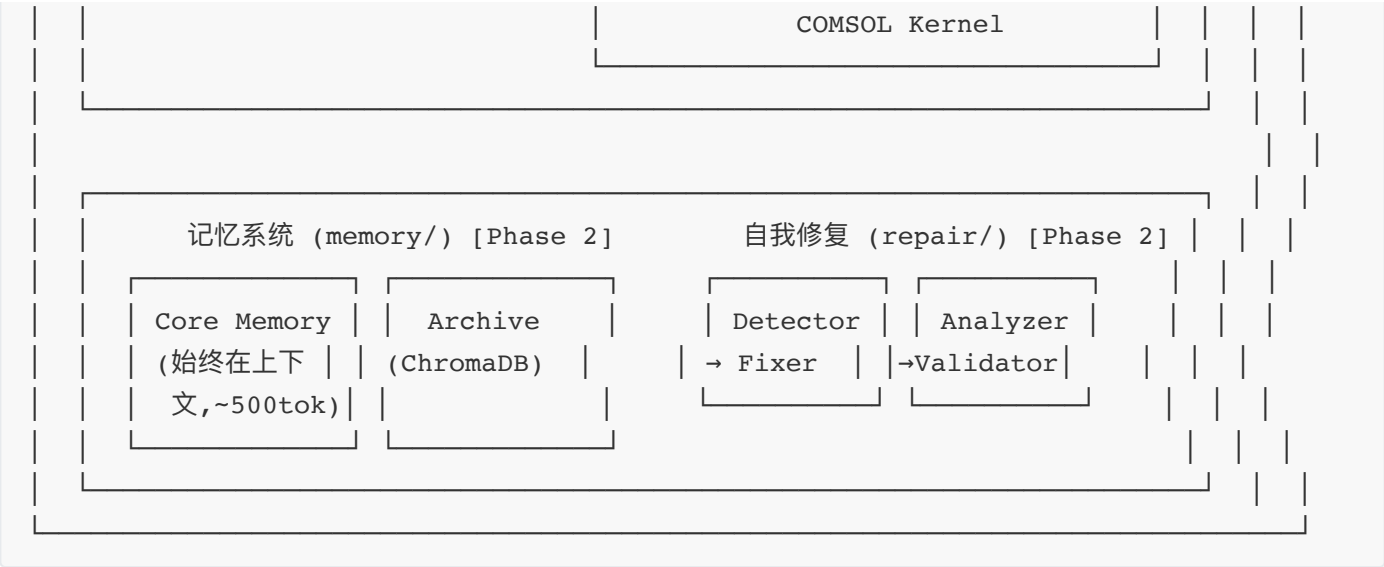
目录

- 1. [系统全景](#)
- 2. [核心数据流](#)
- 3. [技术栈详解](#)
- 4. [Agent 主循环](#)
- 5. [工具系统](#)
- 6. [LLM 抽象层](#)
- 7. [COMSOL 集成层](#)
- 8. [CLI 界面层](#)
- 9. [自我修复系统 \(Phase 2\)](#)
- 10. [记忆与上下文压缩 \(Phase 2\)](#)
- 11. [错误处理策略](#)
- 12. [安全与边界](#)

1. 系统全景

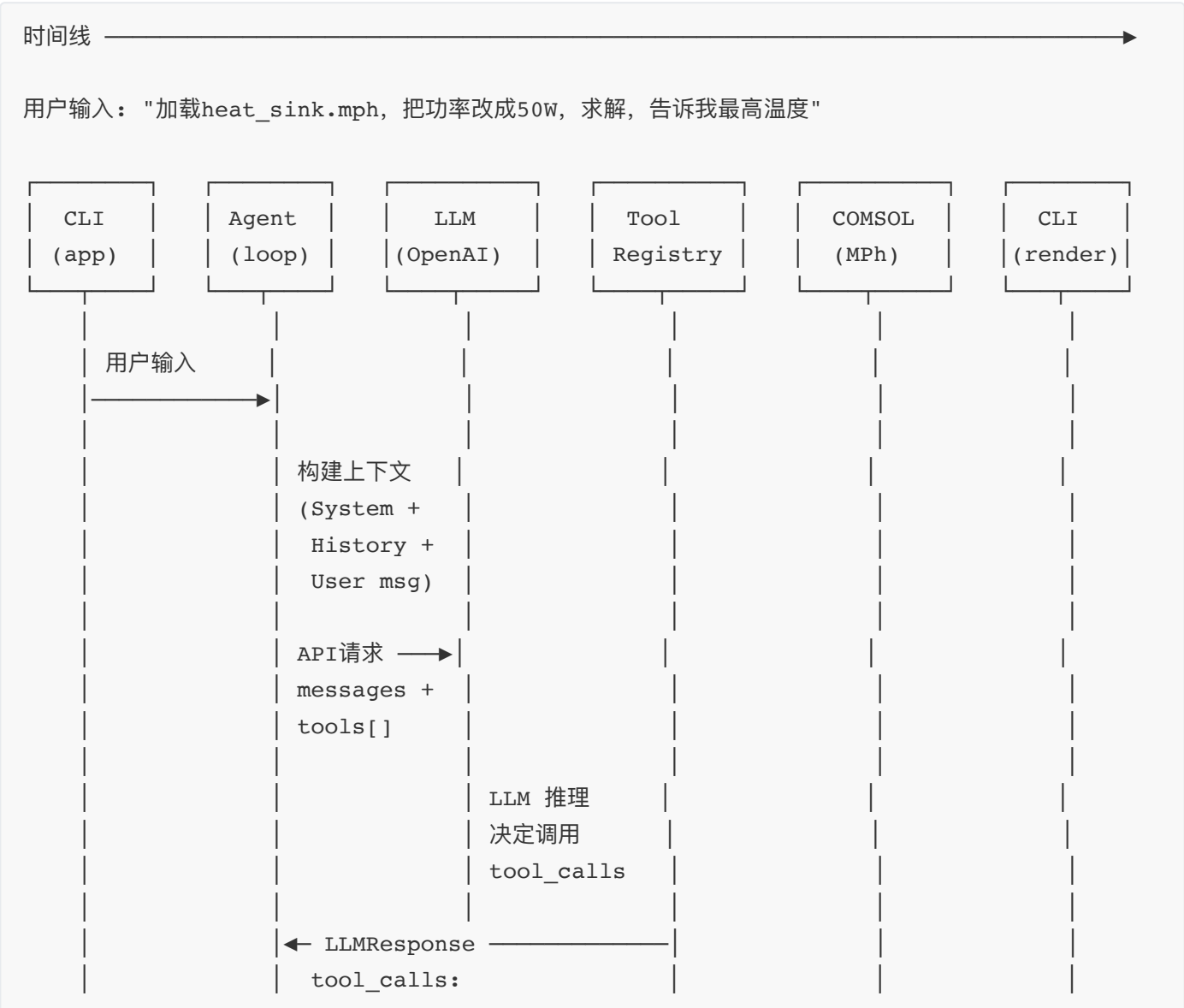


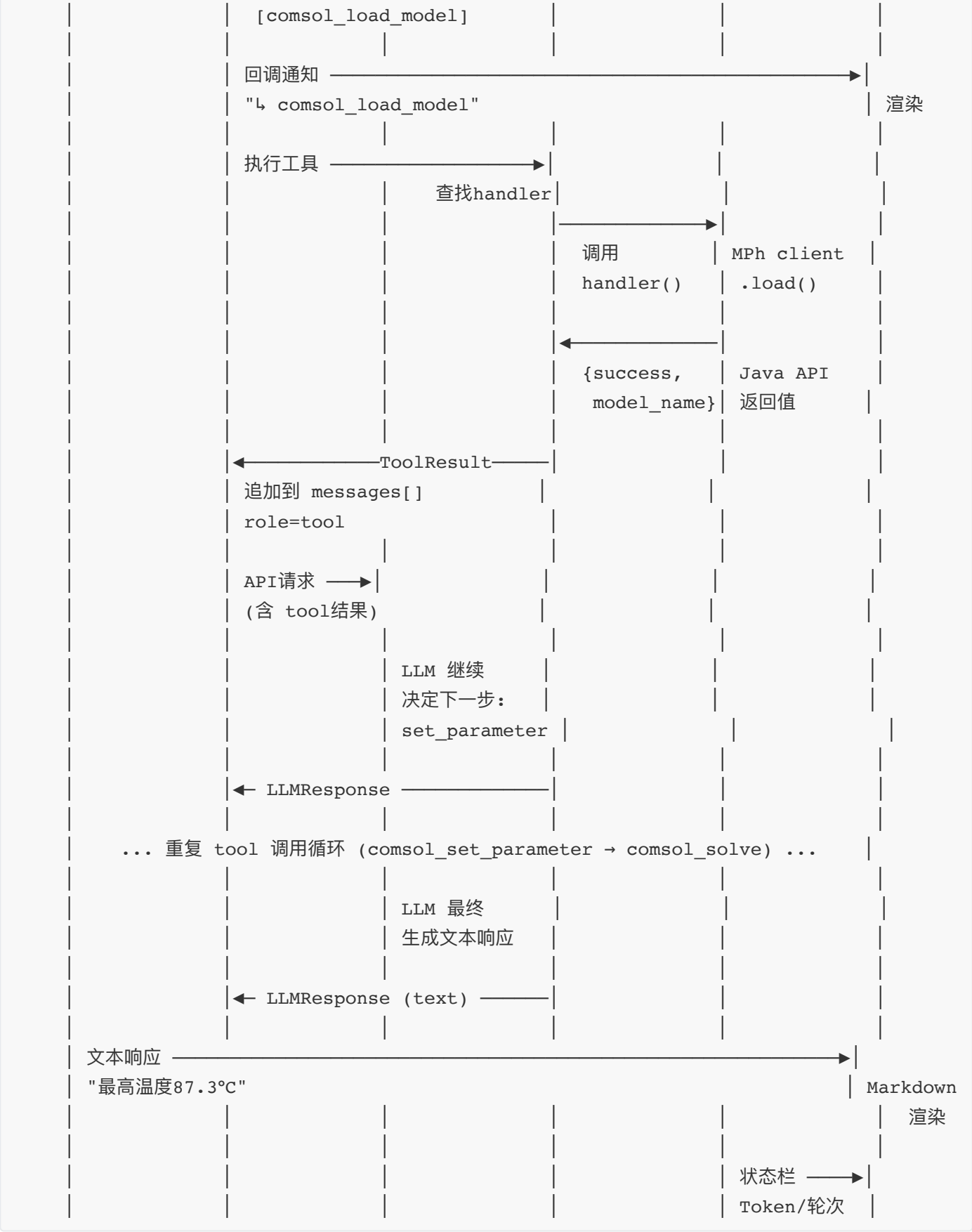




2. 核心数据流

2.1 完整请求处理流程





2.2 消息格式转换流程

系统内部统一使用 **OpenAI 格式** 的消息列表，在调用 Anthropic 时转换：

内部统一格式 (OpenAI-style)

```
{
  "role": "system",
  "content": "You are..."
}

{
  "role": "user",
  "content": "Hello"
}

"...",
{
  "...",
  "role": "assistant",
  {...}}
"content": "Let me help...",
"tool_calls": [
  {
    "id": "call_123",
    "function": {
      "name": "comsol_solve",
      "arguments": "{...}"
    }
  }
]
}

{
  "role": "tool",
  "tool_call_id": "call_123",
  "content": "{...}"
}
```

→
anthropic.py
_convert_messages()

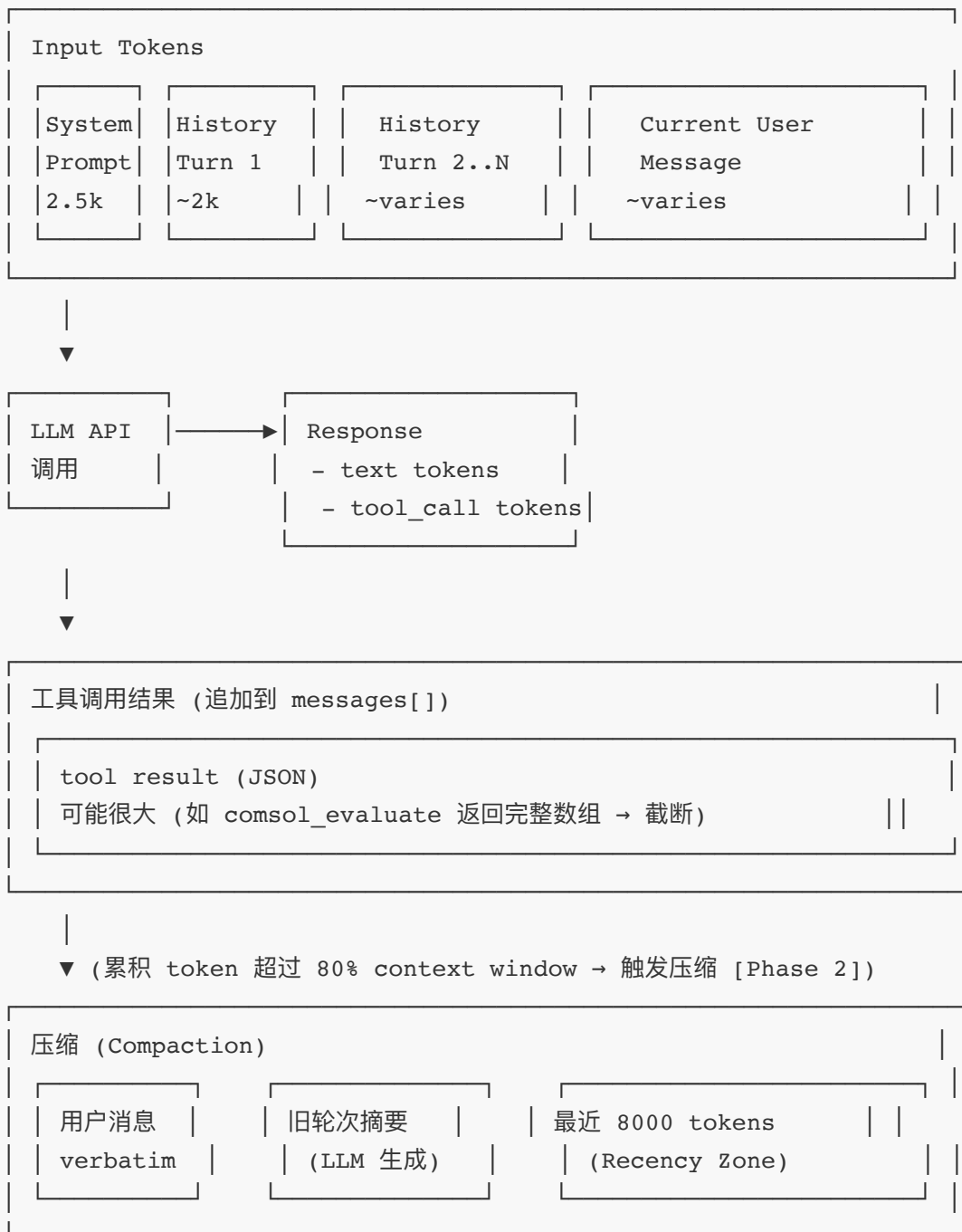
Anthropic 格式

```
system: "You are..."
messages: [
  {
    "role": "user",
    "content": "Hello"
  },
  {
    "role": "assistant",
    "content": [
      {"type": "text", "text":
        {"type": "tool_use", "id":
          "name": "...", "input":
        ]
      },
      {
        "role": "user",
        "content": [
          {"type": "tool_result",
            "tool_use_id": "call_123",
            "content": "{...}"
          }
        ]
      }
    ]
  }
]
```

2.3 Token 生命周期

Session Token 生命周期

每次 LLM 调用:



3. 技术栈详解

3.1 运行时环境

组件	技术	版本	作用
Python	CPython	3.11+ (实测 3.14)	主运行时
Java	JVM (COMSOL 自带)	17+	COMSOL API 运行时
JPyype	Python-Java 桥接	MPh 依赖	让 Python 调用 Java 对象
asyncio	Python 标准库	-	异步 I/O (LLM API 调用)

3.2 依赖分层

第一层：核心运行时	
Python 3.11+, asyncio, dataclasses, pathlib	
第二层：外部通信	
httpx (HTTP), JPyype (JNI)	
第三层：数据与配置	
pyyaml (配置), pydantic (验证), tiktoken (Token计数)	
第四层：LLM SDK	
openai, anthropic	
第五层：COMSOL 桥接	
MPh (包装 JPyype)	
第六层：CLI 渲染	
rich, prompt-toolkit, click	
第七层 (Phase 2)：向量存储	
chromadb, sentence-transformers	
第八层 (可选)：Web UI	
gradio	

3.3 COMSOL 交互三层模型



<code>client.load()</code>	→ 加载 .mph 文件
<code>client.create()</code>	→ 创建空模型
<code>model.parameter()</code>	→ 设置参数 (Pythonic)
<code>model.solve()</code>	→ 求解
<code>model.evaluate()</code>	→ 评估结果表达式 → NumPy 数组
<code>model.export()</code>	→ 导出数据/图片
<code>model.save()</code>	→ 保存 .mph

model.java 属性

▼

Layer 2: Java API (通过 JPyype)

<code>model.java.geom()</code>	→ 几何操作
<code>model.java.geom("geom1").feature().create("cyl",...)</code>	
<code>model.java.physics()</code>	→ 物理场设置
<code>model.java.mesh()</code>	→ 网格生成
<code>model.java.study()</code>	→ 研究/求解器
<code>model.java.sol()</code>	→ 解对象
<code>model.java.resultt()</code>	→ 结果后处理

JNI / 进程间通信

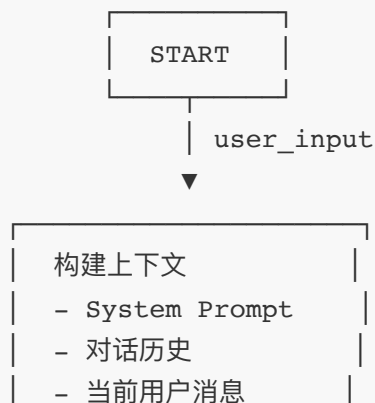
▼

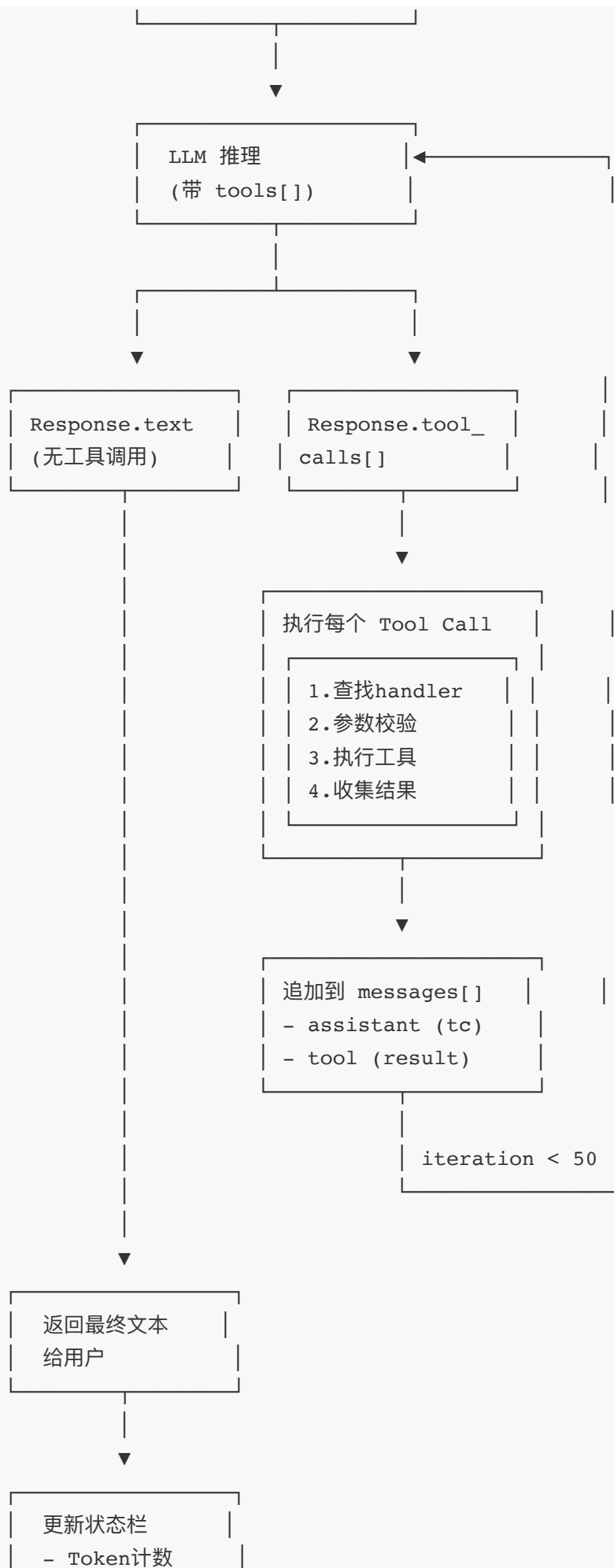
Layer 1: COMSOL 原生进程

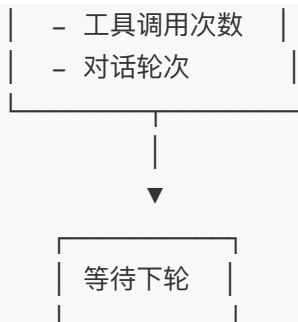
<code>comsol batch -inputfile in.mph -outputfile out.mph -np 4</code> (独立子进程方式, 用于批量/集群场景)

4. Agent 主循环

4.1 状态机







4.2 AgentState 数据结构

```
@dataclass
class AgentState:
    messages: list[dict]          # 完整对话消息 (OpenAI格式)
    turns: list[ConversationTurn] # 结构化轮次记录
    total_tokens_used: int        # 累计 Token 消耗
    tool_iterations_this_turn: int # 当前轮次工具调用次数

# messages 示例:
# [
#   {"role": "system", "content": "You are a COMSOL agent..."},
#   {"role": "user", "content": "加载模型"},
#   {"role": "assistant", "content": "", "tool_calls": [...]},
#   {"role": "tool", "tool_call_id": "1", "name": "comsol_load_model",
#    "content": '{"success": true, ...}'},
#   {"role": "assistant", "content": "模型已加载。最高温度87.3℃。"},
#   {"role": "user", "content": "把材料改成铜"},
#   ...
# ]
```

4.3 关键迭代逻辑

```
conversation = []

FUNCTION process_turn(user_input):
    conversation.append({"role": "user", "content": user_input})

FOR i = 1 TO MAX_ITERATIONS (50):
    // 1. 检查是否需要压缩上下文
    IF estimate_tokens(conversation) > threshold:
        conversation = compact(conversation) // [Phase 2]

    // 2. 调用 LLM
    response = LLM.generate(
        messages=conversation,
```

```

        tools=get_all_tool_definitions()
    )

    // 3. 纯文本响应 → 结束本轮
    IF response.is_text:
        conversation.append({"role": "assistant", "content": response.text})
        RETURN response.text

    // 4. 有工具调用 → 逐个执行
    conversation.append(assistant_tool_call_message(response.tool_calls))

    FOR EACH tool_call IN response.tool_calls:
        handler = lookup_handler(tool_call.name)
        result = handler(**tool_call.arguments) // 执行工具

        conversation.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": serialize(result)
        })

    // 如果工具执行失败 [Phase 2]
    IF result.is_error AND auto_repair_enabled:
        repaired = run_repair_loop(code, error, conversation)
        conversation.append(repaired)

    // 循环继续: LLM 看到 tool 结果, 决定下一步

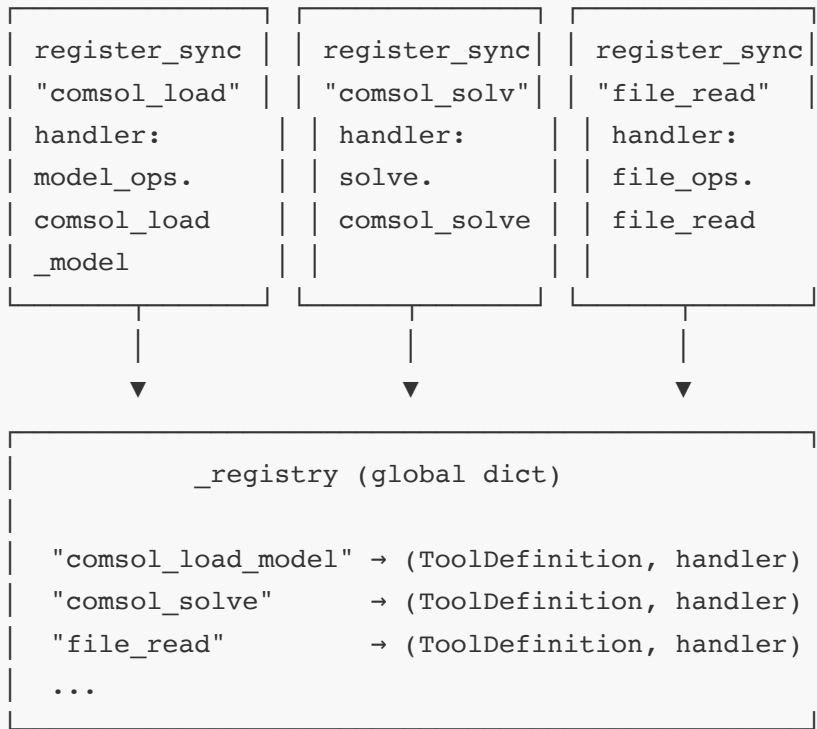
    // 达到最大迭代次数, 强制结束
    RETURN LLM.generate(messages=conversation, tools=[]) // 无工具 → 文本输出

```

5. 工具系统

5.1 工具注册机制





5.2 ToolDefinition 格式 (传递给 LLM)

```
{
  "name": "comsol_solve",
  "description": "Run a COMSOL study/solver on a loaded model. Returns solve status, timing, and convergence information.",
  "parameters": {
    "type": "object",
    "properties": {
      "model_name": {
        "type": "string",
        "description": "Name of the loaded model to solve."
      },
      "study_name": {
        "type": "string",
        "description": "Name of the study to run. Uses the first study if not specified."
      }
    },
    "required": ["model_name"]
  }
}
```

5.3 工具返回格式

所有工具统一返回 dict:

```
# 成功
{"success": True, "model_name": "heat_sink", "elapsed_seconds": 2.1, ...}

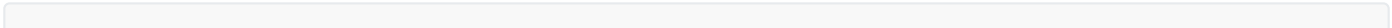
# 失败
{"success": False, "error": "Model 'xxx' not found. Loaded models: ['heat_sink']"}
```

5.4 全部 17 个工具一览

分类	工具名	类型	作用
模型管理	comsol_load_model	读	加载 .mph 文件
	comsol_create_model	写	创建空模型
	comsol_close_model	写	关闭模型
	comsol_save_model	写	保存到磁盘
	comsol_list_models	读	列出已加载模型
	comsol_get_model_summary	读	模型结构摘要
参数	comsol_set_parameter	写	设置参数值
	comsol_list_parameters	读	列出所有参数
求解	comsol_solve	执行	运行研究/求解器
结果	comsol_evaluate	读	评估结果表达式
	comsol_export_results	读	导出 CSV/PNG/.mph
	comsol_plot	读	生成可视化图像
代码	comsol_execute_java	执行	执行 Java API 代码
文件	file_read	读	读文件
	file_write	写	写文件
	file_list	读	列出目录
Shell	shell_execute	执行	执行 shell 命令

6. LLM 抽象层

6.1 类层次结构



LLMProvider (ABC)	ToolDefinition	LLMResponse
└─ generate()	- name: str	- text: str None
└─ generate_stream()	- description: str	- tool_calls:
list[ToolCall]		
└─ count_tokens()	- parameters: dict	- finish_reason: str
		- usage: dict
└─ OpenAIProvider		
└─ client: AsyncOpenAI		ToolCall
└─ _convert_tools() → OpenAI format		- id: str
└─ generate() → AsyncOpenAI.chat.completions.create		- name: str
		- arguments: dict
└─ AnthropicProvider		
└─ client: AsyncAnthropic		ToolResult
└─ _convert_messages() → Anthropic format		- tool_call_id: str
└─ _convert_tools() → Anthropic format		- name: str
└─ generate() → AsyncAnthropic.messages.create		- output: str
		- is_error: bool
└─ LocalProvider (Phase 4)		
└─ Ollama / vLLM		

6.2 OpenAI ↔ Anthropic 消息格式映射

OpenAI		Anthropic
messages: [		messages: [
{role: "system", content: S} →	(extracted to `system` param)	
{role: "user", content: U} →	{role: "user", content: U}	
{role: "assistant",	→ {role: "assistant",	
content: T,	content: [	
tool_calls: [	{type: "text", text: T},	
{id, function: {name, args}}	{type: "tool_use", id, name, input:	
args}	]	
]}	}	
{role: "tool",	→ {role: "user",	
tool_call_id: ID,	content: [	
content: R}	{type: "tool_result",	
	tool_use_id: ID,	
	content: R}	
tools: [	]	
{type: "function",	}	
function: {name, desc,	→ tools: [	
parameters: schema}}	{name, description, input_schema: schema}	
]	]	

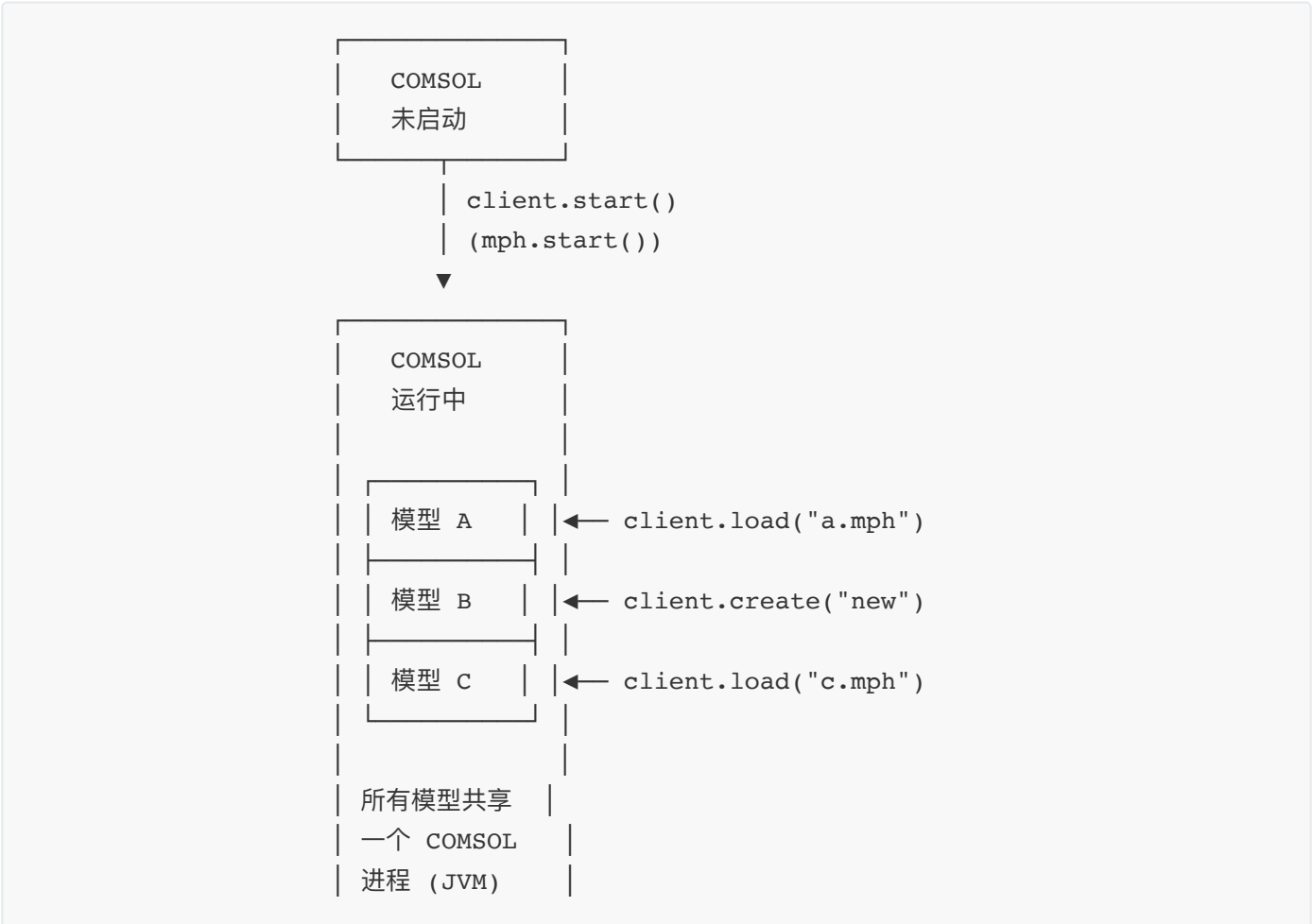
6.3 LLMRouter 路由策略

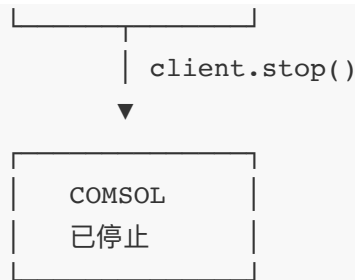
```
# 任务 → 模型路由
ROUTING_RULES = {
    "code_generation":      "strong_model",    # GPT-5 / Claude Opus
    "code_repair":          "strong_model",
    "text_response":        "strong_model",
    "summarization":        "fast_model",       # GPT-5 Mini / Claude Haiku
    "error_classification": "medium_model",
    "memory_compaction":    "fast_model",
}

# 降级策略
FALLBACK_CHAIN = {
    "openai": ["anthropic"],    # OpenAI 不可用 → Anthropic
    "anthropic": ["openai"],    # Anthropic 不可用 → OpenAI
}
```

7. COMSOL 集成层

7.1 MPh 客户端生命周期





7.2 MPh 与 Java API 的调用路径

Python 代码	Java (COMSOL 进程内)
<pre>model.mph_model.parameter()</pre> MPh 包装 自动类型转换 (str→String, NumPy→double[]) ▼ 返回 Python 值 (NumPy array, str, float)	<pre>model.parameter()</pre> 原生 Java API com.comsol.model.Model ▼ 返回 Java 值 (double[], String, int)
<pre>model.java.geom("geom1")</pre> JPyype 桥接 返回 Java 对象代理 ▼ 可链式调用 Java 方法 <pre>.java.geom("g").feature() .create("cyl", "Cylinder")</pre>	<pre>model.geom("geom1")</pre> 直接访问 Java 对象 com.comsol.model.GeomSequence ▼ 可调用所有 Java API 方法 <pre>.geom("g").feature() .create("cyl", "Cylinder")</pre>

7.3 COMSOLClient 单例模式

COMSOLClient (Singleton)
_instance: COMSOLClient None ← 全局唯一 _lock: threading.Lock ← 线程安全 _mph_client: mph.Client None ← MPh 会话 _models: dict[str, ModelHandle] ← 已加载模型注册表
ModelHandle:
name: str ← 模型标识符 path: str None ← .mph 文件路径 java_model: JavaObject ← Java 模型对象 (JPyype 代理) mph_model: mph.Model ← MPh 模型包装


```
is_modified: bool    ← 脏标记
```

8. CLI 界面层

8.1 REPL 循环

```
CLIApp.start()
```

```
1. register_all_tools()    # 注册17个工具
2. create_provider()       # 初始化 LLM
3. AgentLoop(provider)    # 创建 Agent
4. COMSOLClient.start()    # 启动 COMSOL 会话
5. 显示欢迎界面
6. 进入 REPL 循环
```

```
REPL:
```

```
while running:
    input = prompt("> ")          # prompt_toolkit

    if input.startswith("/"):
        result = handle_command(input)
        if result == EXIT:
            break
        continue

    response = await agent.run(input) # Agent 主循环
    render_agent_response(response)  # Rich Markdown
    render_status_bar(...)           # Token/状态
```

8.2 斜杠命令处理

```
输入: "/xxx ..."
```



```
handle_command(command, agent, config)
```

```
/exit, /quit    → CommandResult.EXIT
```

/help	→ render_help()	→ CONTINUE
/clear	→ agent.reset()	→ CLEAR
/compact	→ (Phase 2: 手动触发压缩)	→ CONTINUE
/models	→ 查询 COMSOLClient.models	→ 显示表格
/config	→ 显示当前配置	→ CONTINUE
/memory	→ (Phase 2: 记忆管理)	→ CONTINUE
/skills	→ (Phase 3: 技能列表)	→ CONTINUE
/log	→ 显示最近10条消息	→ CONTINUE
其他	→ 显示错误	→ CONTINUE

8.3 Rich 渲染效果

```
COMSOL Agent v0.1.0
Model: openai/gpt-4o | Session: default
```

```
COMSOL Agent – AI-powered simulation assistant
Provider: openai | Model: gpt-4o
Type /help for commands, /exit to quit.
```

> 加载 heat_sink.mph

🤖 我来加载这个模型...

```
↳ comsol_load_model(filepath='heat_sink.mph')
   Model loaded: heat_sink
```

- 该模型包含：
- 几何: heat_sink_geom
 - 物理场: 固体传热
 - 研究: 稳态热分析

— Token: ██ 2.1k/180k | 本轮工具: 1 | 对话: 2 —

9. 自我修复系统 (Phase 2)

9.1 修复循环架构

代码执行失败
|

修复循环 (Repair Loop)

Phase 1: Detect (detector.py)

输入: tool_result (error=True), error_message

分类逻辑:

错误类型	识别规则
SYNTAX_ERROR	"Syntax error", "unexpected token"
API_ERROR	"No method", "NullPointer", "ClassCastException"
PHYSICS_ERROR	"Failed to find", "not defined", "boundary condition"
SOLVER_ERROR	"Failed to converge", "NaN", "singular matrix", "diverged"
TIMEOUT_ERROR	"timed out", "exceeded"
UNKNOWN	未能匹配以上任何模式

输出: ErrorReport(type, message, code_snippet, context)



Phase 2: Analyze (analyzer.py)

输入: ErrorReport + 对话历史 + COMSOL API 索引 (RAG)

LLM 分析 Prompt:

"You are debugging a COMSOL simulation error.

Error type: {type}

Error message: {message}

Related code: {snippet}

Relevant API docs: {retrieved_docs}

Identify the root cause and propose a fix approach."

输出: Diagnosis(root_cause, fix_strategy, api_hints)

▼

Phase 3: Fix (fixer.py)

输入: Diagnosis + 原始代码 + 约束

修复策略 (按错误类型):

L1 (SYNTAX): 直接修正语法

- 补全括号/分号
- 修正类型声明
- 修正方法签名

L2 (API): 查询 API 文档 + 修正

- 替换为正确的 API 调用
- 补全缺失的参数
- 修正参数类型

L3 (PHYSICS/SOLVER): 调整物理/数值参数

- 细化网格
- 调整求解器容差
- 修改边界条件
- 切换求解器类型

LLM 修复 Prompt:

"Given this COMSOL code: {original_code}

Root cause: {diagnosis}

Fix strategy: {strategy}

Generate the corrected code.

Rules:

1. Minimal change to fix the error
2. Preserve the original physics intent
3. Use correct COMSOL API calls
4. Return ONLY the corrected code"

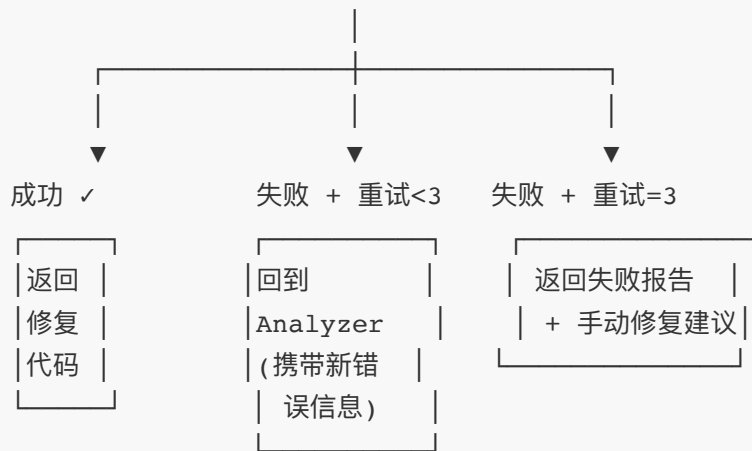
输出: FixedCode(code, explanation, confidence)

▼

Phase 4: Validate (validator.py)

1. 重新执行修复后的代码
2. 检查是否仍有相同错误
3. 如果是 SOLVER_ERROR, 检查结果物理合理性:
 - 温度范围合理吗? (如 0-5000K)

- 应力不超过材料强度?
 - 质量/能量守恒?
4. 返回 `ValidationResult(success, new_error, warnings)`



9.2 修复循环的对话上下文注入

修复过程会追加到 agent messages 中:

```
[
  ... (之前的对话) ...
  {
    "role": "tool",
    "tool_call_id": "tc_123",
    "content": '{"success": false, "error": "Failed to converge..."}'
  },
  // ↓ 修复系统自动注入 ↓
  {
    "role": "user",
    "content": "[SYSTEM] The previous tool call failed. Auto-repair is
analyzing..."
  },
  {
    "role": "assistant",
    "content": "Diagnosis: Mesh too coarse near the boundary...",
    "tool_calls": [{"name": "comsol_execute_java", "arguments": {"java_code":
"..."}}]
  },
  // 修复后的 Java 代码执行结果
  {
    "role": "tool",
    "content": '{"success": true, "output": "Converged in 5 iterations"}'
  }
]
```


第3层: Archive (持久化存储, ~/.comsol_agent/archive/)

SQLite 表:

```
sessions: id, name, created_at, summary
memories: id, session_id, type, content, embedding
templates: id, name, domain, java_code, params
```

ChromaDB 集合:

```
memory_embeddings: 语义检索历史对话
comsol_api_docs: COMSOL API 文档向量索引
simulation_templates: 仿真模板向量索引
```

10.2 压缩触发与执行流程

压缩触发条件 (任一满足):

1. Token 使用 $> \text{context_window} * \text{compaction_threshold}$ (默认 80%)
2. 用户手动 `/compact`
3. 新 session 启动时

压缩流程:

Step 1: 评估

```
total_tokens = estimate_messages_tokens(messages)
recency_tokens = 估算最近 8000 tokens
target_tokens = recency_tokens + old_user_messages
to_compress = total_tokens - target_tokens
```



Step 2: 分组

```
groups = group_by_turn(messages[system:]....)
        → [(turns 1-5), (turns 6-8), ...]

for each group outside recency zone:
    extract user_messages (verbatim, keep)
    extract tool_results (truncate > 2000 chars)
    extract assistant_replies (to be summarized)
```

Step 3: 逐组压缩 (增量)

```
// 顺序处理, 每组基于前组摘要
living_summary = ""
for group in old_groups:
    summary = FAST_LLM.summarize(
        previous_summary=living_summary,
        new_turns=group,
        template="""
        Previous context: {living_summary}
        New turns: {group}

        Create an updated summary. Include:
        1. Models loaded/modified
        2. Key parameter values
        3. Simulation results (key numbers only)
        4. Errors encountered and how fixed
        5. User preferences revealed
        Limit: 500 tokens max.
        """)
    living_summary = summary
```

Step 4: 重建消息列表

```
new_messages = [
    system_prompt,
    {"role": "user", "content": "[Context: {summary}]"},
    ...old_user_messages, // verbatim
    ...recency_zone_messages,
]
```

Step 5: 持久化到 Archive

```
// 生成结构化记忆卡片
memory_card = STRONG_LLM.summarize(
    turns=compressed_turns,
    template="Extract key facts as structured JSON:
    {model_params: {...}, results: {...},
    decisions: [...], preferences: [...]}"
```



```

    )

    // 写入 ChromaDB (用于语义检索)
    chroma_collection.add(
        ids=[f"session_{id}_block_{n}"],
        embeddings=[embed(memory_card)],
        metadatas=[{session_id, turn_range, summary}],
        documents=[memory_card]
    )

    // 写入 SQLite (用于时间线查询)
    db.execute(
        "INSERT INTO memories (session_id, content, token_count)"
        "VALUES (?, ?, ?)", (sid, memory_card, tokens)
    )

```

10.3 记忆检索 (新 Session 启动 / 用户查询)

当用户提到之前讨论过的内容：

用户： "上次那个铝板模型，把尺寸改成20x10cm"

```

1. 语义检索
   query_embedding = embed("铝板模型 尺寸 铝板 heat")
   results = chroma_collection.query(
       query_embeddings=[query_embedding],
       n_results=5
   )

2. 相关性过滤
   relevant = [r for r in results if r.score > 0.7]

3. 注入上下文
   context_msg = {
       "role": "user",
       "content": f"[Retrieved from memory: {r.content}]"
   }
   messages.insert(after_system, context_msg)

```

11. 错误处理策略

11.1 分层错误处理

Level 1：用户输入层

错误：空输入、无效命令	
处理：静默忽略 / 显示帮助信息	
恢复：继续 REPL	

Level 2：LLM 通信层

错误：API 超时、Rate Limit、认证失败、网络错误	
处理：重试 (exponential backoff: 1s, 2s, 4s, max 3次)	
降级：OpenAI → Anthropic (如果配置了)	
用户：显示清晰错误信息 + 建议	
恢复：如果重试成功 → 继续；失败 → 返回错误给用户	

Level 3：工具执行层

错误：COMSOL 未启动、模型未找到、参数错误、求解发散	
处理：返回 {"success": False, "error": "..."}	
反馈：LLM 看到错误信息，可以建议替代方案	
修复：[Phase 2] 触发自我修复流程	
恢复：LLM 根据错误类型决定下一步	

Level 4：系统层

错误：内存不足、磁盘满、COMSOL 进程崩溃	
处理：记录日志；尝试优雅关闭 COMSOL 会话	
用户：显示严重错误 + 建议重启	
恢复：需要用户干预	

11.2 LLM API 重试策略

```
async def generate_with_retry(messages, tools, max_retries=3):
    for attempt in range(max_retries):
        try:
            return await provider.generate(messages, tools)
        except RateLimitError:
            wait = 2 ** attempt # 1s, 2s, 4s
            await asyncio.sleep(wait)
            # 如果使用了 OpenAI, 尝试切换到 Anthropic
```

```

        if attempt == max_retries - 1 and fallback_provider:
            provider = fallback_provider
    except (APITimeoutError, APIConnectionError):
        wait = 2 ** attempt
        await asyncio.sleep(wait)
    except AuthenticationError:
        raise # 不重试, 直接报错

raise MaxRetriesExceeded("LLM API call failed after 3 retries")

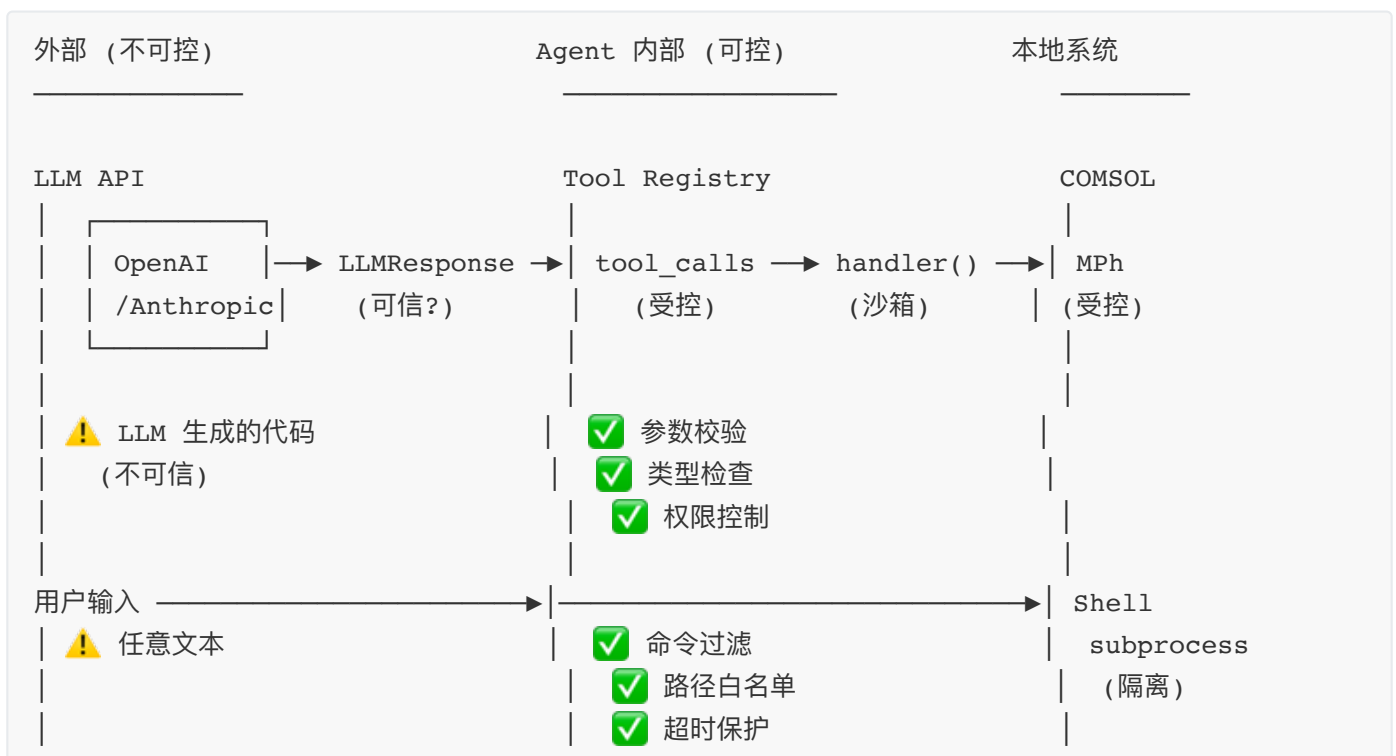
```

12. 安全与边界

12.1 安全限制

维度	限制	原因
Shell 执行	timeout ≤ 600s; 禁止 <code>sudo / rm -rf /</code> 模式	防止破坏性操作
文件操作	仅限用户工作目录 + 配置的路径	防止读取敏感文件
COMSOL	仅操作已加载的模型	防止意外修改其他模型
LLM Token	单轮最大 50 次工具调用	防止无限循环
内存	工具输出截断到 10,000 字符	防止上下文溢出

12.2 数据流中的边界



12.3 审计追踪

```
~/comsol_agent/
├─ logs/
│   └─ agent.log          # 完整执行日志
├─ sessions/
│   └─ <timestamp>.json  # 对话历史备份
└─ config.yaml           # 配置 (不含 API key)
```

附录 A: 当前代码文件清单

```
comsol_agent/
├─ __init__.py           # 版本定义
├─ main.py               # CLI 入口 (click)
├─ cli/
│   ├── app.py           # REPL 循环, prompt_toolkit
│   ├── commands.py      # 斜杠命令处理
│   ├── config.py        # YAML 配置 + 环境变量
│   └─ renderer.py       # Rich 终端渲染
├─ agent/
│   ├── loop.py          # Agent 主循环
│   ├── prompt.py        # 系统提示词构建
│   ├── tool_registry.py # 工具注册/查找
│   └─ tools_bootstrap.py # 注册全部17个工具
├─ tools/
│   ├── file_ops.py      # 文件+Shell 工具
│   └─ comsol/
│       ├── client.py    # COMSOL 客户端 (MPh, 单例)
│       ├── model_ops.py # 模型管理工具
│       ├── solve.py     # 求解+评估+Java执行工具
│       └─ evaluate.py   # 导出+绘图工具
├─ llm/
│   ├── base.py          # LLMProvider ABC, ToolDefinition, LLMResponse
│   ├── openai.py        # OpenAI GPT provider
│   ├── anthropic.py     # Anthropic Claude provider
│   └─ router.py         # Provider 创建工厂
├─ memory/               # [Phase 2]
├─ repair/               # [Phase 2]
├─ simulation/           # [Phase 3]
└─ utils/
    ├── token_counter.py # tiktoken 封装
    └─ logger.py         # 日志工具
```

附录 B: 关键接口契约

AgentLoop.run()

输入: `user_input: str`
输出: `final_response: str`

副作用:

- 修改 `self.state.messages` (追加对话历史)
- 修改 `self.state.total_tokens_used`
- 可能触发 COMSOL 操作 (加载/修改/求解模型)

异常:

- 不会抛出未捕获异常 (内部错误转为文本响应)

COMSOLClient (Singleton)

```
get_instance() → COMSOLClient
start(cores?, version?) → None
stop() → None
load(filepath) → ModelHandle
create(name) → ModelHandle
get_model(name) → ModelHandle # raises KeyError
save(name, filepath?) → str (saved path)
close(name, save?) → None
execute_java(java_code, model_name?) → str
get_model_summary(name) → str
```

Tool Handler

签名: `async def handler(**kwargs) → dict`

返回格式:

成功: `{"success": True, <tool-specific fields>}`
失败: `{"success": False, "error": "<human-readable message>"}`

约束:

- 必须是 `async` (同步工具用 `register_sync` 包装)
- 参数通过 `**kwargs` 接收 (由 JSON Schema 约束类型)
- 返回值会被 JSON 序列化